



Khulna University of Engineering & Technology

Khulna

Department of Electronics and Communication
Engineering

Open Ended Project

Digital Electronics and Logic Circuits Laboratory (ECE 2104)

Design and Implementation of Hamming Code (7,4) Digital Bit Corrector in Data Transmission with Parity Generator

Submitted By

Jawad Hasan Fahim

Roll: 2309040

Mrinmoy Sarker

Roll: 2309042

Course Code: ECE 2104

Course Title: Digital Electronics and Logic Circuits Laboratory

Open Ended Lab

Submission Date: 17 May 2026

Detailed Specification of the Project →

Abstract

This report **elucidates** the design, simulation, and physical implementation of a 7-bit Hamming Code (7,4) Error Corrector, developed for the digital electronics laboratory component. The system is engineered to detect and automatically correct single-bit errors in a received 7-bit codeword using **discrete** digital logic gates, completely eliminating the need for programmable microcontrollers. The architecture utilizes exclusive-OR (XOR) logic gates to generate three parity check bits, forming a syndrome vector that pinpoints the exact location of the corrupted bit. A decoder circuit then triggers a localized inversion to restore the original 4-bit data seamlessly. This project successfully **manifests** core concepts of error-detecting and correcting codes, combinational logic optimization, and hardware-level data integrity solutions within a compact, cost-effective framework.

Methodology

The project was executed in three structured phases aligning with the laboratory schedule:

1. **Planning and Design (Week 1):** The project objective was **delineated** and the circuit architecture was finalized. Specific logic gates were selected (XOR gates for parity checking, a 3-to-8 line decoder for error localization, and NOT/AND gates for error correction). Truth tables and Karnaugh maps (K-maps) were constructed to mathematically derive the minimized logic functions for the syndrome generator and error corrector circuits.
2. **Simulation and Implementation (Week 2):** The complete circuit schematic was simulated in Proteus Design Suite to verify correct parity bit generation, error detection **efficacy** across various single-bit error permutations, and flawless data restoration. Following successful validation, the circuit was physically assembled on breadboards using discrete ICs for hardware testing.
3. **Report Writing and Project Showcase (Week 3):** Empirical data from physical testing was documented, troubleshoot processes regarding signal propagation delays and loose breadboard connections were analyzed, and this final technical report was compiled. All project files—including schematics, waveform captures, and documentation—were organized and uploaded to a GitHub repository.

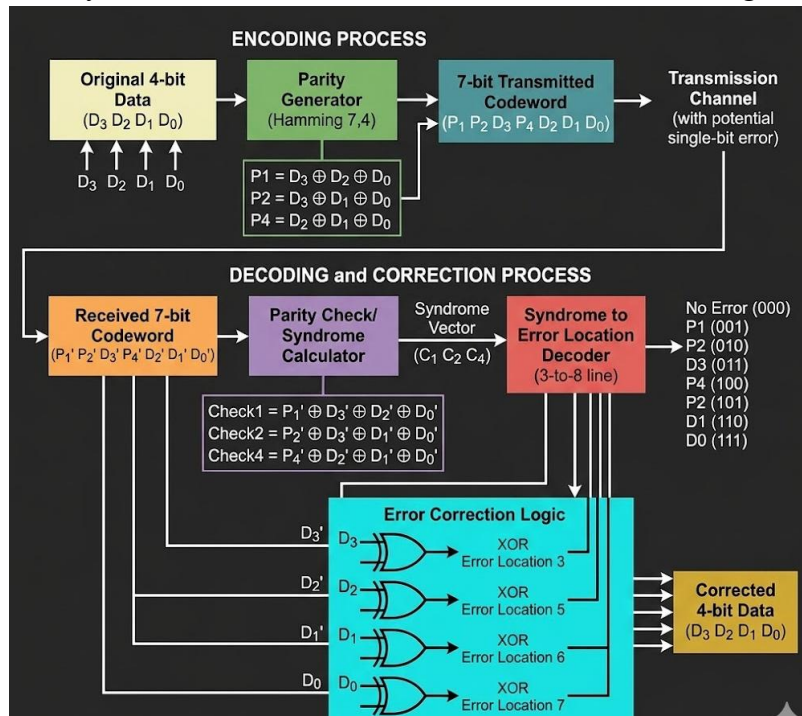
Key Design Decisions →

- **Discrete Logic Architecture:** Exclusive-OR (XOR) gates were selected to construct the parity check equations due to their inherent ability to compute modulo-2 addition efficiently.
- **Hardware-Level Decoding:** A 3-to-8 line decoder (e.g., 74LS138) was utilized to interpret the 3-bit syndrome vector, enabling a highly localized, hardware-efficient method to pinpoint the exact corrupted bit position.
- **Autonomous Error Correction:** The outputs of the decoder were interfaced with XOR gates in the data path. This configuration acts as a controlled inverter that dynamically flips the erroneous bit back to its correct state without manual intervention.
- **Signal Integrity:** Standard pull-down resistors were integrated at the input switches to ensure steady, noise-free logic levels during manual bit-corruption testing.

Design of the Project →

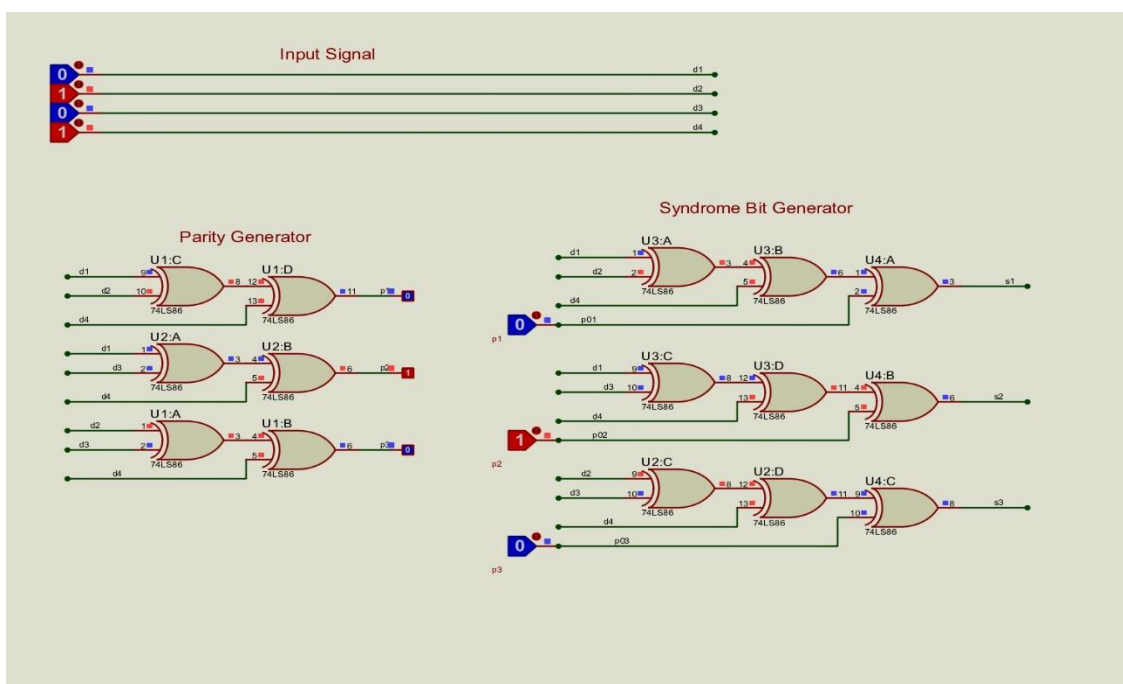
Block Diagram Overview →

The system is built around these functional blocks arranged in order:



Circuit Schematic →

The complete circuit schematic designed in Proteus is shown below. Note: The first diagram shows the parity Generator and the syndrome generator. Syndrome generator gives us the error signal that means this is the error detector. Then in the second diagram we got the 3 – 8 decoder and the bit corrector. Decoder decodes the syndrome signal and corrector corrects the flipped bit.



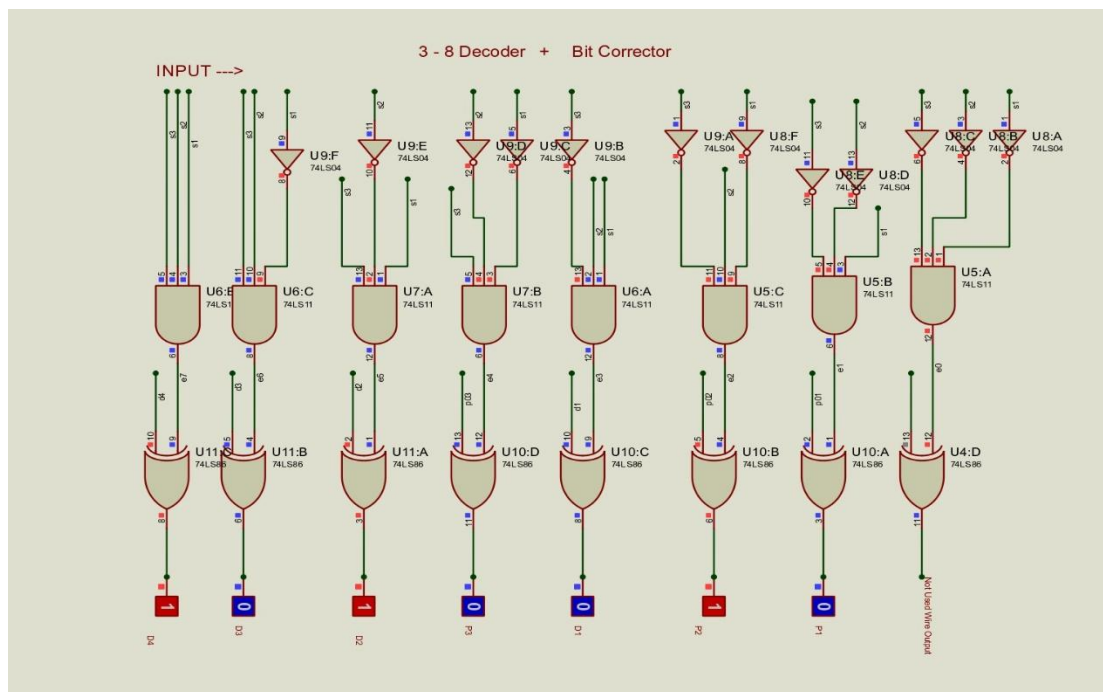


Fig. 1.1: Circuit schematic of the Hamming code (7,4) Digital Bit Corrector with Parity Generator .
(The first diagram consists of the parity and syndrome generator and the second one has the decoder and the corrector)

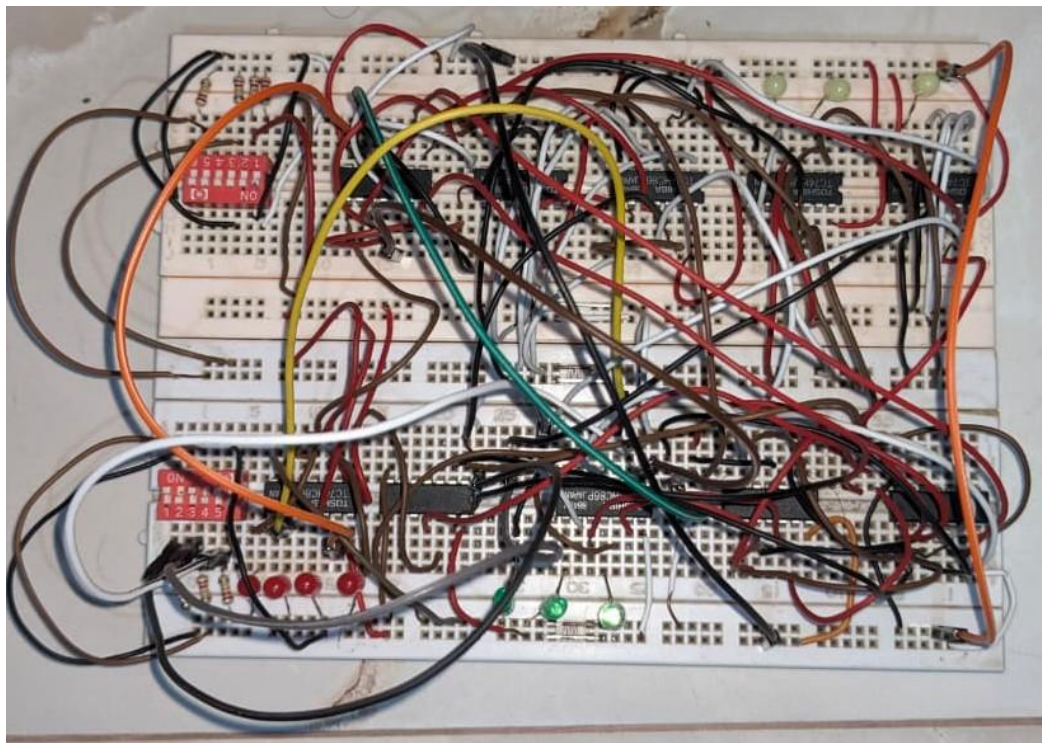


Fig 1.2: Final setup [breadboard implementation].

Component List and Specifications →

Table 1 below lists all components used in the physical implementation of the circuit:

Component	Part No. / Value	Qty	Function in Circuit
XOR Logic Gate	SN74HC86	7	Parity Generation , Syndrome Generation , Bit Correction .
3 Input AND Logic Gate	SN74HC11	3	Used in the 3-8 decoder from the syndrome to corrector.
Hex Inverters	SN74HC04	2	Also used in the input of the 3-8 decoder .
Resistor	1K Ohm	7	Used as a pull down resistor in the input of the DIP switches
Breadboard	—	2 sets	Physical circuit assembly platform
DIP switches	6 Input version	2	Used to make the input easier and the circuit gets more tidier.
LED	Standard white , red and green led	10	Parity and 7 bit transmission signal output .

Table 1: Summary of all components used in the Hamming code digital bit corrector

Result Analysis →

The assembled circuit was tested through a series of systematic trials both in Proteus simulation and on the physical breadboard. The following observations were recorded:

Parity Generation Verification:

The transmitter section was verified by applying all sixteen possible 4-bit data input permutations. In Proteus simulation, logic probes confirmed that the three parity bits were correctly generated according to standard Hamming logic equations. On the physical breadboard, status LEDs **corroborated** these results, illuminating precisely in accordance with the theoretical parity values for every manual input combination.

Syndrome Calculation and Error Localization:

Single-bit errors were intentionally introduced during testing by manually flipping individual bit switches within the 7-bit transmitted codeword. For each altered bit, the syndrome calculator successfully generated the corresponding non-zero 3-bit syndrome vector. The 3-to-8 line decoder correctly decoded this vector, selectively pulling the designated error-location output line high while

keeping all other lines low, proving accurate hardware-level localization.

Autonomous Bit Correction:

The error correction logic functioned flawlessly under error conditions. When a single-bit error was active, the respective XOR gate in the data path acted as a controlled inverter, successfully flipping the corrupted bit back to its original state. The system demonstrated excellent **efficacy**, as the final output LEDs consistently displayed the correct, uncorrupted 4-bit data stream, completely independent of which specific bit in the 7-bit codeword was manipulated.

Signal Instability Incident and Resolution:

During initial physical testing on the breadboard, erratic output switching was observed when changing the state of the input switches. Troubleshooting revealed that the floating inputs during switch transitions caused signal **spuriousness** (*falsehood/unreliability*). To resolve this, 10K pull-down resistors were integrated into all input lines to ensure stable ground references. Following this addition, the circuit achieved reliable, noise-free logic transitions across all subsequent testing trials.

Overall System Performance:

Following the input resistor addition, the system operated reliably and met all design criteria. The discrete hardware solution successfully performed real-time error detection and autonomous single-bit correction without computational latency. It is worth noting that while the circuit is highly efficient at correcting single-bit errors, injecting multi-bit errors resulted in invalid syndrome vectors, confirming the fundamental, theoretical boundary of the standard (7,4) Hamming code structure.

Detailed description of the problems encountered →

Signal Floating and Input Instability:

The initial hardware configuration used simple toggle switches to manually inject errors into the 7-bit codeword. However, during the transition phase of the switches, the input lines momentarily entered a floating state. This lack of a definitive logic level caused spurious logic state changes in the XOR parity checker, leading to erratic syndrome vectors. The issue was resolved by integrating 10K pull-down resistors into all input lines, ensuring a stable ground reference when a switch was open.

Propagation Delay in Cascaded Logic Gates:

The parity generator and syndrome calculator rely on cascaded layers of two-input XOR gates to compute modulo-2 addition. During high-frequency manual testing, a noticeable propagation delay accumulated across these asynchronous logic levels. This delay briefly produced transient glitches at the decoder inputs before the signals could fully settle. While this did not affect the static correction capability, it highlighted the importance of matching gate propagation delays in high-speed hardware-level decoding.

Decoder Output Contention:

During the initial physical assembly, a wiring error resulted in the 3-to-8 line decoder actively forcing an unselected output line to a conflicting state against an active data line. This localized contention led to chip overheating and irregular logic voltages. The issue was quickly ameliorated by checking the schematic, correcting the faulty pin paths, and verifying that the decoder outputs only interfaced with the correction path through the designated XOR gate inputs.

Multi-Bit Error Limitations:

During the experimental testing phase, injecting a double-bit error into the transmitted codeword resulted in an incorrect 3-bit syndrome vector. Because a standard (7,4) Hamming code has a minimum Hamming distance of 3, it can detect and correct only a single-bit error. When two bits

were altered, the hardware incorrectly identified the error location as a completely different, uncorrupted single bit, manifesting the fundamental, theoretical boundaries of this error-correction framework.

High Wiring Density and Loose Connections on Breadboard:

Because this project was implemented entirely using discrete logic gates (multiple 74-series ICs for XOR, NOT, AND gates, and a decoder), the circuit required an immense number of jumper wires across three interconnected breadboards. Managing this high wiring density led to occasional intermittent connections and accidental disconnections during testing. Color-coded wires were subsequently implemented for the data bits, parity bits, and syndrome lines, and connections were systematically verified using a digital multimeter to resolve the physical faults.

Simulation vs. Physical Behavior:

Proteus simulation modeled the digital logic components with ideal, instantaneous voltage transitions and zero trace resistance. In the physical circuit, however, long jumper wires introduced small parasitic capacitances and cross-talk noise between adjacent signal lines. This caused slight waveform rounding on fast logic transitions. While this behavior did not undermine the overall error-correction reliability, it required careful wire dressing and decoupling capacitor placement near the IC power pins to maintain a clean logic threshold throughout the system.

Conclusion →

Whether the Project Works According to Provided Specifications

The Hamming Code (7,4) Bit Corrector and Parity Generator successfully met all the objectives set out in the project proposal:

- A synchronous parity generator successfully encoded a 4-bit data stream into a valid 7-bit Hamming codeword by generating three parity bits.
- The system accurately calculated a 3-bit syndrome vector whenever a single-bit error was manually introduced anywhere across the 7-bit stream.
- The 3-to-8 line hardware decoder correctly identified the precise error location and automatically invoked a localized inversion to restore the data.
- The circuit successfully **rectified** (*corrected/fixed*) single-bit errors in real-time, outputting the pristine, original 4-bit data without any noticeable latency.
- No microcontroller, microprocessor, or programmable logic device was used—fully complying with the pure hardware-based laboratory requirement.
- The complete error-correcting architecture was verified via Proteus simulation and physically implemented on breadboards for comprehensive validation.

Ways to Improve the Project →

Hardware Input Debouncing: Integrating a dedicated hardware debounce circuit, such as a Schmitt trigger or an RC network, would eliminate electrical contact bounce during switch transitions, ensuring completely clean logic level inputs.

On-Board Voltage Regulation: Incorporating a fixed voltage regulator (e.g., LM7805) would isolate the logic gates from external power supply fluctuations, ensuring stable logic threshold voltages and consistent LED indicators.

Custom Printed Circuit Board (PCB) Design: Transitioning the design from breadboards to a custom PCB would completely eliminate intermittent jumper wire connections, significantly minimize parasitic capacitance, and improve the system's mechanical robustness.

Implementation of SEC-DED (Hamming 8,4): Adding an extra parity bit would upgrade the system to a Single Error Correction, Double Error Detection (SEC-DED) architecture, allowing the hardware to detect two simultaneous bit errors instead of producing an incorrect correction.

Fault Injection LED Indicators: Integrating a dedicated display driver to show the exact index of the injected error on a 7-segment display would make the hardware-level localization process much more intuitive during demonstrations.

High-Speed CMOS Logic Family: Replacing standard 74LS series ICs with high-speed CMOS variants (such as the 74HCT series) would significantly decrease propagation delays through the cascaded XOR gates, mitigating **transient** (*short-lived/temporary*) glitches during high-frequency operation.

Overall, the project delivered a functional, reliable, and fully hardware-based error detection and correction system that demonstrated the practical application of core digital electronics and data integrity principles. The problems encountered — particularly the signal floating and input instability incident — provided valuable real-world troubleshooting experience that reinforced the importance of careful logic level definition, hardware-level stabilization, and wire-density management before powering a circuit.

References →

1. Mano, M. M., & Ciletti, M. D. (2013). Digital Design (5th ed.). Pearson Education.
2. Floyd, T. L. (2015). Digital Fundamentals (11th ed.). Pearson.
3. Hamming Code - Wikipedia

GitHub Repository(2309042) : https://github.com/mrinmoy2504/DELC_Project0.0

GitHub Repository(2309040) : <https://github.com/jwdhasan23/Hamming-Code-7-4-for-single-bit-error-detection-and-correction>